VLA Generalization for low level continuous control game environments

Meeting notes:
VLA Meeting Summary
VLA Project - Discussion with Zhiding
VLA generalization
VLA generalization slides
Meeting Notes Zihan/Talha 2/2
Meeting Notes 2/10
2/18 latency-VLA meeting
3/28 VLA meeting nvidia

https://docs.google.com/document/d/1pkOUUwC3kQLK1TmALCCAWD54P9KT-ZWWwEGgLm4x2jA/edit?tab=t.0#heading=h.8bawxo412rp7
(latency storyline)
https://chatgpt.com/c/69d6a1bc-c86c-832e-8690-d45b475be88b

Post-Submission TODOs (May 12):

TASK1: Readme

1. Install (one-line) [Talha]
    a. install.sh

2. Small model training **(done)**
    a. small_models/train.py \ **[Xinyuan]**
        env.name=flappy_bird \
        env.config=FLAPPY_BIRD_CONFIG \
        train.max_steps=xxx \
        train.batch_size=xxx \
        wandb_project=xxx \
        wandb_experiment=xxx \
        output_path=OUTPUT_PATH

b. small_models/rollout.py \ **[Zeju]**
    env.name=flappy_bird \
    env.config=FLAPPY_BIRD_CONFIG \
    model.config=xxx \
    model.path=xxx \
    rollout_path=xxx \
    rollout_config.num_of_prompts=xxx \
    wandb_project=xxx \
    wandb_experiment=xxx
    …
c. small_models/rollout_realtime.py / rollout_realtime bash \ **[Xinyuan]**

d. scripts/small_models/XXX.sh **[Xinyuan: training; Talha: push trajs to HF]**
    lots of bash aligned with codebase

e. small_models/plots/XXX.py **[Zeju: Heatmap, Talha: architecture]**

3. VLA training
    vla/train.py **[Talha]**
        [INFRA] Sub-process evaluation
    vla/rollout/ **[Talha]**
        openvla.py
        pi0.py
        pi05.py
    vla/rollout_async/ **[Xinyuan]**
        openvla.py
        pi0.py
        pi05.py
    scripts/vla/XXX.sh **[Talha, Xinyuan, Zeju]**

vla/plots/XXX.py **[Zeju: Heatmap, Xinyuan: async + profile table, Talha: vla 0-latency vs mix-latency]**

4. Demo **[Talha, Xinyuan] PUT FILES ONLY, NO NEED TO MERGE**

demo/flappy_bird.py

demo/xxx.py

5. Assets, configs

6. Remove TB logs and replace with wandb_logs/ **[Talha]**

wandb_logs/xxx.md

In each md: list the link to corresponding experiments

Figure 1: wandb.ai/xxxx

7. latency_bench **[Xinyuan]**

1. Scripts for data mixing from different model rollouts are not ready **[Talha]**

DATA_REGIME: (each json / jsonl)

[

[{“Observation”: xxx, “Action”: xxx, “Step”: xxx, “Reward”: xxx},{“Observation”: xxx, “Action”: xxx, “Step”: xxx, “Reward”: xxx},{“Observation”: xxx, “Action”: xxx, “Step”: xxx, “Reward”: xxx},],

[..]

]

Data mixing:

Source: file1, file2, file3, file4

Mixing ratio: ratio1, ratio2, ratio3, ratio4

Total data: 10000

Shuffle: true

(curriculum: shuffle=False, and source starts from latency0 to latency5; Training: also set shuffle=False)

Experiment Todos:

1. Small model RL:
    a. Memory (RNN, LSTM, Transformer) + Latency injection (no latency, channel, late fusion) sweep across all environments
2. VLA:
    a. **Zero latency:** PI-0, Pi-0.5, Gr00t training for all environments
    b. **Single latency:** OpenVLA, Pi-0, PI-0.5, Gr00t [Pick any 3 latencies]
    c. **Mixed latency:** OpenVLA, Pi-0, Pi-0.5, Groot [Curriculum (gradually increase the latency for eps), Random (randomly shuffled episodes with varying latency)]
    d. **Cross task:** AxB across environments, all models/one model
    e. **Jitter training:** all environments, all models, fixed jitter setting []
    f. Jitter Eval
    g. GPU latency profiles

1. Zero shot VLM inference across all the environments, all models - all latency
2. Small models on all the environments, all models - all latency
    a. Training on one latency from env, eval across all latency
    b. Training using distilled latency from teacher, eval across all
3. Zero shot VLA inference across the environments, all models - all latency

4. SFT trained VLA across all the environments, all models - all latency
5. RL latency trained VLA across all the environments, 1/2 models - all latency

Motivation about project:

- **Core motivation:** Action spaces for VLAs: are they generalizable? If not, we can claim that we work on VLMs that can produce generalizable action spaces (token space), and think about how to optimize them. And also: training will be easier
- **Key idea 1:** Benchmark on VLMs with high-frequency tasks (with NL tokens)
- **Optimizing VLMs:**
  - Perception: Draw a plot [input resolution vs performance]
  - Generation: think vs no-think
  - General: evaluate perception vs generation time
- **Formulation:**
  - Previous formulation: turn-based MDP
  - Now: incorporating time in MDP
  - (Simulation for Real-time flows: only record time for vLLM generation, and send it to the environment to track how much time goes. Verify the result with single-vLLM thread so no batch-related latency is introduced)

Models:

VLMs:

1. Qwen-3-VL

VLAs:

1. SmolVLA
2. CombatVLA [Trained for games specifically]
   a. https://www.lumine-ai.org/[game vla]
      -> may need system prompt to map keyboard actions to game actions

3. OpenVLA
4. Pi 0.5
5. Nitrogen
6. https://craftjarvis.github.io/JarvisVLA/

Environments GAME related

a. Mujoco (embodied / robotics, including Luna Lander)
b. VizDoom
2. Latency-sensitive-bench
3. Car Racing in Gym
4. Flappy Bird [Talha]
5. Slime Gym [zihan]
6. Cooperative Pong
7. MiniWorld

OTHERS

RTRBench for robotics [add newer benchmarks for robotics]
Thodoroff's real-time RL benchmark
rtgym (real-time gym wrapper)
CARLA for driving (realistic, but maybe not latency-sensitive)
Sample Factory for high-frequency simulation (optional)

Method:

1. [fixed-time interval] Proceed a fix amount of time each turn, agent keeps action during the proceeding time [tune hyperparameter for proceeding time t] [Talha]
2. [single-thread processing] Using non-batch processing and record time, and let env proceed that much time [Zihan]

Related Work:

1. SIMA
2. SIMA 2
3. Sim2Real Transfer for Deep Reinforcement Learning with Stochastic State Transition Delays

4. Reinforcement Learning for Control Systems with Time Delays: A Comprehensive Survey
5. https://openreview.net/pdf?id=Z8UfDs4J46
6. https://arxiv.org/abs/2406.03102
7. BELIEF-BASED OFFLINE REINFORCEMENT LEARNING FOR DELAY-ROBUST POLICY OPTIMIZATION
8. Directly Forecasting Belief for Reinforcement Learning with Delays

Focus for now:

- What off-the-shelf VLAs we can evaluate on VizDoom
    - Action space for VLAs - generalizable? [Talha]
    - [NO] SmolVLA: continuous actions, may not generalize to a different robot having different actions. Not sure if this is enough for ALL possible envs to test on.
    - CombatVLA: discrete actions, but generalizable, since it is semantic tokens. Not sure if it can be treated as VLA generalization in a broader sense.
    - OpenVLA: discrete tokens directly from LLM, but output tokens TIED to robot kinematics, since it has an action detokenizer.
    - Pi 0.5: promising, since converts high level instr to actions, but low level actions still env specific
- Other than VLLM / SGlang, what infra do current VLAs use for faster generation [Talha]
    - Training-Time Action Conditioning for Efficient Real-Time Chunking
    - Real-Time Execution of Action Chunking Flow Policies
    - No standardized method, inference a hot topic for VLAs. -> we can use vllm / SGlang for now
- How do they manage memory if executed in long-terms [Talha]
- VLMs for game envs [Talha]
- VizDoom implementation for RAGEN / VAGEN / Any other Infra [Talha: check feasibility for RAGEN/VAGEN]
- RAGEN/VAGEN: implement async for inference / training [Zihan]

Start from latency sensitive generalization
VLMs as VLAs are not designed for specific tasks, better generalization

- Have both experiments

Environments:

1. **Flappy bird, Demon Attack (other 2d environment)**
2. **3D environment** (Minecraft, open-ended, eval challenge; Other environments of 3D, lightweight?)
3. **Long-horizon environment (https://github.com/Farama-Foundation/ViZDoom)**

1. RL baselines with/without latency
   https://arxiv.org/pdf/2106.09678 **Secant baseline**
   shadow
   Larger MLP
   Transformers
   RNNs

   Latency distribution:
   1. Within episode
   2. Each episode with fixed latency
   3. Hidden latency
2. Zero shot performance VLAs (Inference/ no training needed)
3. Zero latency trained VLAs performance across latency settings (total = l)
   a. Choose which models - (choose 3)
   b. Choose which environments to train on - (choose 3 simulated, 1 real world environment)

   Total expts: a x b for training. Then (a x b) x l for inference

S0, a0, s1, a1, s2, a2

Latency = 2

Give S0
Make sure a0, a1 is Noop
Supervision: a2

S1 -> a3
S2 -> a4

4. Latency training for VLAs:
   RL infra: https://github.com/RLinf/RLinf
   a. Generalized training: Training data generalized across latencies
      i. With one model, b environments and then benchmark across all b environments with all l latencies?
      ii. With all models? a models across b environments and then benchmark across all b environments with all l latencies?
   b. Latency aware training for latency: Training for one latency?
      i. Train across latency l, eval across all other latencies??

Mar 26 Meeting
https://www.youtube.com/watch?v=OBoQyRoeGzU
Demon Attack Demo
https://research.nvidia.com/person/mingjie-liu
Mingjie Liu, Verilog Expert
https://arxiv.org/pdf/2106.09678
Secant, RL Baseline